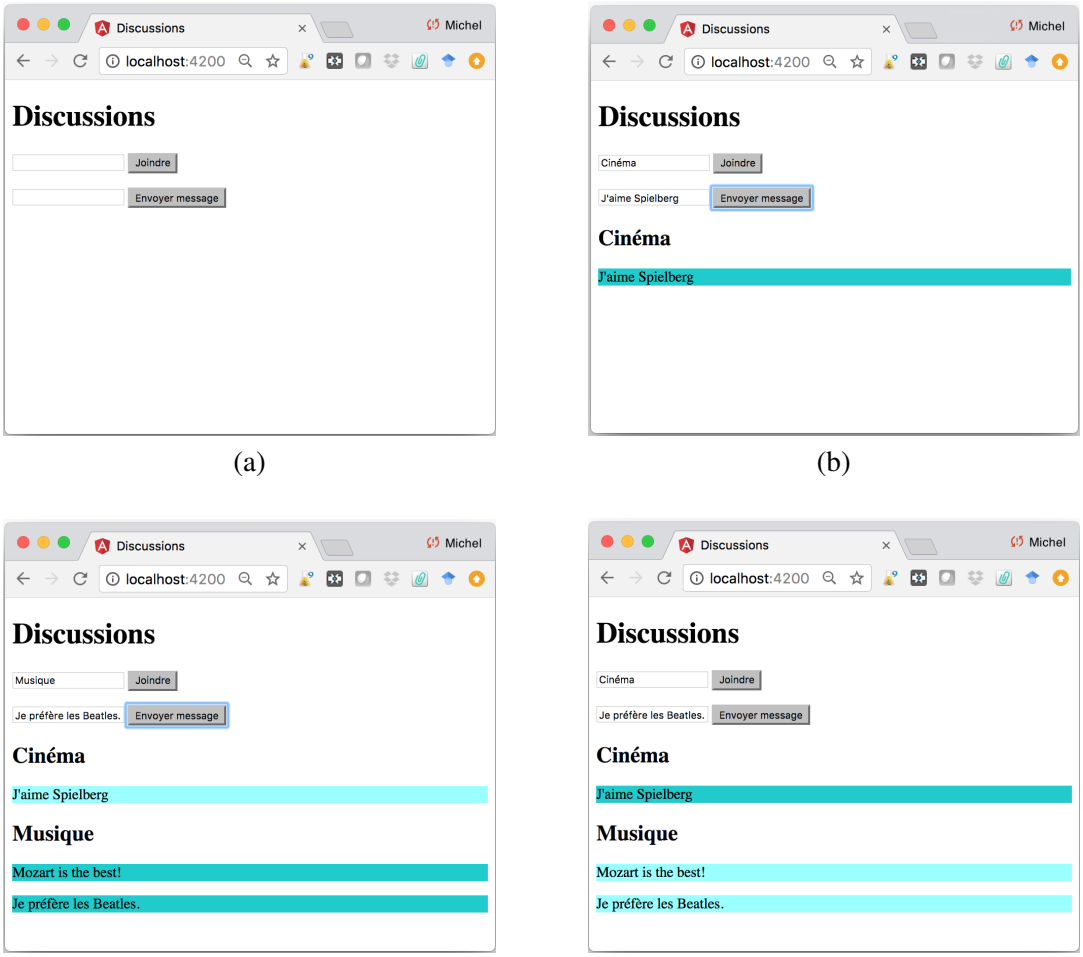
FIGURE 1 – Application avec Angular et Socket.io



1 Plateforme MVC sur le client et Socket.io (5 points)

On désire implémenter une application web qui permet de gérer des discussions sur plusieurs thèmes différents entre plusieurs utilisateurs. Lorsqu'on accède au site, on obtient la vue illustrée à la figure **??**a. Dans la première boîte, on peut écrire un thème. Lorsqu'on clique sur le bouton «Joindre», on se connecte sur le serveur sur une «room» correspondant à ce thème. Celle-ci sera créée si elle n'existe pas déjà. Sinon on se joint à tous les autres utilisateurs connectés à cette «room». Chaque fois qu'un utilisateur écrit un message dans la seconde boîte et qu'il clique sur le bouton «Envoyer message», le message est diffusé à tous les utilisateurs qui ont joint le thème qui apparaît dans la première boîte. Le message s'ajoutera à la liste de tous les messages qu'on a déjà reçus sur ce thème. À la figure **??**b, l'utilisateur s'est enregistré au thème «Cinéma» et a envoyé un message qui vient de s'afficher comme premier de la liste associée à ce thème.

La figure **??**c illustre l'état obtenu un peu plus tard : après que l'utilisateur se soit inscrit au thème «Musique», un autre utilisateur a envoyé un message sur ce thème, et notre utilisateur a ensuite ajouté un nouveau message sur ce même thème. On voit donc dans l'affichage tous les messages qui ont été envoyés, pour chaque thème, depuis le moment où notre utilisateur s'est joint à ce thème. À noter que les messages liés au thème courant, soit celui écrit dans la boîte supérieure, utilisent une couleur de fond plus foncée. Si on clique sur un des messages d'un thème, c'est celui-ci qui devient le thème courant : il s'affiche alors dans la boîte supérieure et ce sont tous les messages de ce thème qui prennent alors une couleur de fond plus foncée. La figure **??**d illustre ce cas : on a cliqué sur le message de la section «Cinéma», et c'est ce thème qui devient courant.

On vous demande d'implémenter cette application en utilisant Angular et Sokcet.io. Vous compléterez le code qui suit :

Fichier app.component.ts (la documentation du conteneur Map est fournie en annexe) :

```
import { Component } from '@angular/core';
import * as io from 'socket.io-client';

@Component({
    selector: 'app-root',
    templateUrl: './app.component.html',
    styleUrls: ['./app.component.css']
})
export class AppComponent {
    topicList: Map<string,string[]>;
    socket: SocketIOClient.Socket;
    topic: string;

    public ngOnInit(): void {
        // À compléter
    }

    // À compléter
}
```

Fichier app.component.html (à modifier) :

```
<h1>Discussions</h1>
<p>
  <input>
  <button>Joindre</button>
</p>
<p>
  <input>
  <button>Envoyer message</button>
</p>

<div >
  <h2> Thème</h2>
  <p> Un message </p>
</div>
```

Fichier app.component.css (pas nécessaire de le modifier) :

```
p.message {
    background-color: #99ffff;
}

div.current p {
    background-color: #00cccc;
}

// Remarque: #00cccc est plus foncé que #99ffff
```

Code sur le serveur :

```
let app = require('express')();
let http = require('http').Server(app);
let io = require('socket.io')(http);
let port =  8000;

// À compléter

http.listen(port, function(){
  console.log('listening on *:' + port);
});


// app.component.html:
<h1>Discussions</h1>
<p>
  <input [(ngModel)]="topic">
  <button (click)="join(this.topic)">Joindre</button>
</p>
<p>
  <input [(ngModel)]="message">
  <button (click)="send(this.topic,this.message)">Envoyer message</button>
</p>

<div *ngFor="let t of topicList.keys()"  [ngClass]="{current: t === this.topic}">
  <h2> {{t}}</h2>
  <p *ngFor="let message of topicList.get(t)"
      class="message" (click)="selectTopic(t)" > {{message}} </p>
</div>

// app.component.ts:
export class AppComponent {
    topicList: Map<string,string[]>;
    socket;
    topic: string;

    public ngOnInit() {
        this.topicList = new Map();
        this.socket = io('http://localhost:8000');
    }

    public join(topic) {
        this.socket.emit('enter',topic);
```

```
        this.socket.on(topic,(message) => {
               if (!this.topicList.has(topic)){
                   this.topicList.set(topic,[message]);
               }
               else {
                  let mList = this.topicList.get(topic);
                  mList.push(message);
                  this.topicList.set(topic,mList);
              }
      });
    }

    public selectTopic(topic) {
       this.topic = topic;
    }

    public send(topic,message) {
        this.socket.emit(topic,message);
    }
}

// Code sur le serveur:
io.on('connection', function(socket){
    let topicList = [];
    socket.on('enter',
          (topic) => {
          socket.join(topic);
          if (!topicList.includes(topic)) {
              topicList.push(topic);
              socket.on(topic,
                (message) => { io.to(topic).emit(topic,message)});
          }
      });
});
```

2 Infographie avec three.js (4 points)

a) (1,5 point) Expliquez comment animer une voiture avec Angular et ThreeJS à partir de l'appui d'une touche jusqu'au relâchement de celle-ci. (Qu'arrive-t-il lorsque la touche «w» est enfoncée et relâchée une seconde plus tard ?)

Vous pouvez supposer que :

— Maintenir la touche «w» enfoncée fait accélérer le véhicule.

— Le véhicule ralentit lorsqu'aucune touche n'est enfoncée.

— Vous n'avez pas besoin de mentionner la gestion des collisions.

Appui de la touche : Un HostListener capture l'événement et appelle une fonction qui vérifie qu'il s'agit de la touche pour faire avancer le véhicule. Celle-ci met à jour une variable booléenne pour indiquer que le véhicule devrait avancer.
Début de la boucle de jeu

— Si le booléen est vrai, ajouter une force d'accélération sur le véhicule. Sinon, appliquer une force de frottement pour le faire décélérer.

— Mettre à jour la position et la vitesse du véhicule

— Afficher la nouvelle position du véhicule

Fin de la boucle de jeu
Lorsque la touche est relâchée, un HostListener capture l'événement et modifie la variable booléenne qui indique que le véhicule est en accélération.

b) (1,5 point) Expliquez de manière concise comment on peut détecter une collision entre deux voitures avec ThreeJS.

D'abord, on détecte une possibilité de collision dès que la distance entre les points centraux de deux voitures est inférieure à la longueur d'une voiture. Si c'est le cas, on déclence la détection de collision. Pour ce faire, il y a plusieurs approches possibles. On peut faire divers lancers de rayon à partir du centre la boîte de délimitation de l'objet vers chaque arête de cette boîte. Pour chaque lancer, on prend le premier objet de la pile d'objets traversés par le rayon et si sa distance est inférieure à celle de l'arête, une collision est détectée.

c) (1 point) Que fait le code suivant :

```
import * as THREE from 'three';

class Camera extends THREE.PerspectiveCamera {
   private position: THREE.Vector3;

   constructor() {
      super();
      this.position = new THREE.Vector3(0,0,100);
   }

   public mystere(position: THREE.Vector3): void {
```

```
        this.lookAt(position);
    }
}

let camera = new Camera();
camera.mystere(new THREE.Vector3(50,50,0));
```

Il crée une caméra placée en haut au centre et qui est orientée vers un coin du plan

3 Programmation asynchrone (1 point)

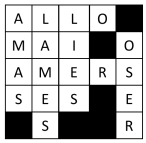
Soit le code suivant, pour solliciter la création d'une grille de mots croisés à un service REST.

```
public creerGrille(info: InfoGrille): Promise<Identificateur> {
     return this.http.post(URL_SERVICE,info)
                     .toPromise()
                     .then(identificateur => this.decode(identificateur))
                     .catch(() => Promise.reject("Erreur"))
}
```

Expliquez ce que fait ce code.

On demande service http de Angular d'exécuter une requête POST. L'observable reçu et transformé en promesse. Si la promesse est tenue, on retourne une nouvelle promesse avec l'identificateur décodé. Sinon, on retourne une promesse rejetée.

FIGURE 2 – Exemple de grille de mots croisés



4 Serveur et MongoDB (3 points)

Soit une base de données MongoDB qui contient des grilles de mots croisés. Chaque grille a un identificateur et une liste de mots. Pour chaque mot, on indique les numéros de rangée et de colonne où il débute et son orientation (horizontale ou verticale). Ainsi, la grille de la figure **??** serait représentée de la manière suivante :

```
{ "id" : 122,
   "words" : [
       { "row" : 1, "col" : 1, "orient" : "h", "word" : "allo" },
       { "row" : 2, "col" : 1, "orient" : "h", "word" : "mai" },
       { "row" : 3, "col" : 1, "orient" : "h", "word" : "amers" },
       { "row" : 4, "col" : 1, "orient" : "h", "word" : "ses" },
       { "row" : 1, "col" : 1, "orient" : "v", "word" : "amas" },
```

```
        { "row" : 1, "col" : 2, "orient" : "v", "word" : "lame" },
        { "row" : 1, "col" : 3, "orient" : "v", "word" : "lies" },
        { "row" : 2, "col" : 5, "orient" : "v", "word" : "oser" } ] }
```

Voici maintenant un extrait d'un service web logé sur le serveur `www.service.org` qui permet de gérer cette base de données :

```
app.put(
    '/crossword/:id/:row/:col/:orient/:word',
    (req, res) => client.connect('mongodb://wwww.service.org:27017/data')
                    .then((db) => db.collection('crosswords')
                                    .update({id: parseInt(req.params.id),
                                             "words.row": parseInt(req.params.row),
                                             "words.col": parseInt(req.params.col),
                                             "words.orient": req.params.orient},
                                              $set: {"words.$.word": req.params.word}}))
                    .then(() => res.send('ok'))
                    .catch((err) => res.send("Erreur:" + err)));
```

a) (1 point) Expliquez ce que fait ce code et fournissez un exemple d'appel à ce service. *Remarque* : l'opérateur «$» identifie l'élément courant du tableau.

Il remplace un mot de la grille par un autre. Pour changer AMERS par AMENS, on l'appelle en faisant la requête http suivante : PUT www.service.org/crossword/122/3/1/AMENS.

b) (1 point) Ce code correspond à un «middleware» en Express. Expliquez de manière claire et concise ce concept fondamental en Express.

Un middleware permet de traiter une requête et de former une réponse. On spécifie une route qui détermine le patron de requête qu'il traite. On peut spécifier le type de requête (GET, POST, etc) ou il peut être générique est s'appliquer à tous les types de requêtes. Lorsque son traitement est terminé, il peut envoyer la réponse, ou passer au prochain middlewarre.

c) (0,5 point) Expliquez pourquoi, pour implémenter ce middleware, la méthode «post()» n'a pas été utilisée.

Parce qu'ici on veut modifier un item déjà existant. Un POST est utilisé pour ajouter un nouvel item.

d) (0,5 point) Décrivez brièvement un autre type d'appel que l'on pourrait vouloir implémenter pour ce service web.

Un appel GET, qui permet de retourner le mot à une position donnée.

5 Programmation en Typescript (5 points)

Soit une grille de mots croisés de dimension 4×4 et ne contenant aucune case noire. Supposez que vous avez accès à un lexique de la forme suivante :

```
lexique = [ "ABAT", "ABBE", "ABEE", "ABER", ...] ;
```

On veut implémenter en Typescript une classe Grid qui représente une telle grille. Voici une ébauche de la classe :

```
const DIMENSION = 4;
const MAX_MUTATION = 10000;

class Grid {
   private rows: string[] = [];
   private columns: string[] = [];

   constructor(private lexicon: string[]) {
      // À compléter
   };

   public checkGrid(): boolean {
      // À compléter
   }

   public show(): void {
      this.rows.forEach((word: string) => console.log(word));
   }

   private fill(): void{
      // À compléter
   }

   private randomInteger(range: number): number {
      return Math.floor(Math.random() * range);
   }

   private selectWord(): string {
      return this.lexicon[this.randomInteger(this.lexicon.length)];
   }

   private checkWords(list1: string[], list2: string[]): boolean {
      for (let i = 0; i < DIMENSION; i++){
         for (let j = 0; j < DIMENSION; j++) {
              if (list1[i][j] !== list2[j][i])
                 return false;
              }
          }
      return true;
   }

   private mutate(): void {
      // À compléter
   }
}
```

Lorsque le constructeur de la classe est appelé, une grille valide doit être construite. Pour y arriver, on choisit aléatoirement un mot du lexique pour chaque rangée et chaque colonne. Ensuite, tant qu'on n'a pas une grille valide, on crée une mutation de la grille. Une mutation consiste à choisir aléatoirement un mot de la grille et le remplacer par un autre mot du lexique. Pour éviter d'entrer dans une boucle infinie, on détermine un nombre maximal de mutations.

a) (3 points) Complétez le code aux endroits indiqués. Assurez-vous que votre code soit de bonne qualité.

```
constructor(private lexicon: string[]) {
   for (let i = 0; i < DIMENSION; i++){
       this.rows.push(this.selectWord());
       this.columns.push(this.selectWord());
   }
   this.fill();
};

public checkGrid(): boolean {
   return this.checkWords(this.rows,this.columns);
}

private fill(): void {
   let i = 0;
   while (!this.checkGrid() && i < MAX_MUTATION) {
      this.mutate();
      i += 1;
   }
}

private mutate(): void {
  let list = this.randomInteger(2) === 0 ? this.rows : this.columns;
  let index = this.randomInteger(DIMENSION);
  let newWord = this.selectWord();
  list[index] = newWord;
}
```

b) (1 point) Qu'y a-t-il de particulier avec le paramètre `lexicon` du constructeur ?

Il déclare de manière implicite un attribut privé de la classe.

c) (1 point) Il y a deux particularités de Typescript qui font en sorte que l'implémentation de la méthode `show()` est très compacte. De quoi s'agit-il ?

La méthode forEach permet d'éviter l'écriture d'une boucle. La fonction fléchée est plus concise qu'une déclaration normale de fonction.

6 Intégration continue (1 point)

Lorsqu'on fait un «commit», il faut s'assurer que le nombre de lignes de code concernées soit minimal. Pourquoi ?

S'il y a un bogue, c'est plus facile de trouver sa source.

7 Assurance qualité (1 point)

Vous savez que les tests unitaires sont importants pour l'assurance qualité. Encore faut-il savoir bien les utiliser. Citez trois critères importants d'une bonne utilisation des tests unitaires.

— Tests écrits avant le code.
— Couverture complète de la fonction et de toutes les situations possibles.
— Intégration du test au projet en même temps que la fonction.
— Le code du test doit lui aussi être de bonne qualité.
— Lors de chaque «push», il faut s'assurer que tous les tests sont exécutés avec succès.

8 Question bonus (0,5 point)

Voici ce qu'on dit sur Wikipédia au sujet des mots croisés :

> C'est l'Anglais Arthur Wynne qui, par l'invention des cases noires, permit au jeu de se développer pleinement, en permettant la dissymétrie lignes/colonnes et en multipliant à l'infini les combinaisons (formes variées des grilles, croisements multiples, mots de longueurs différentes, présence possible de plusieurs mots par ligne ou par colonne, etc.). Sa première grille fut publiée le 21 décembre _____ dans le supplément du New York World, le Fun.

Sauriez-vous identifier l'année de création du premier mots croisés ?

a) 1913
b) 1926
c) 1934
d) 1946
e) 1949

1913

Annexe - conteneur Map()

L'objet Map représente un dictionnaire, autrement dit une carte de clés/valeurs. N'importe quelle valeur valable en JavaScript (que ce soit les objets ou les valeurs de types primitifs) peut être utilisée comme clé ou comme valeur.
On peut savoir le nombre d'items qu'il contient par le biais de son attribut *size*.
Son interface est composée des méthodes suivantes :

clear() Supprime tous les items qu'il contient.

set(clé,item) Associe un item à une clé.

delete(clé) Retire l'item associé à la clé.

get(clé) Permet d'obtenir l'item associé à la clé.

has(clé) Indique si la clé existe dans le Map.